LECTURE 10

Object Detection

From Classification to Localisation

IoU · NMS · YOLOv8

Binh Nguyen — School of Mathematics and Statistics

DATA305 / DATA475 · Week 9 · Google Colab ready

TODAY'S QUESTION

"What objects are in this image, and where are they?"

CLASSIFICATION — what we covered (L1–L9)

Image → Backbone → Dense head → one class label

DETECTION — today

Image → Backbone → Detection head → N × (box, class, confidence)

Inference-first lecture · No training required · YOLOv8n on built-in images

Today's Agenda & Learning Objectives

Four sections · inference-first · YOLOv8n on built-in images

1

Why Object Detection?

Classification vs detection · vision-task taxonomy

2

Boxes · IoU · NMS

Bounding boxes · IoU · confidence threshold · NMS

3

Detection Architectures

Two-stage (R-CNN family) · one-stage (YOLO) · anchors

4

Evaluation & Demo

Precision · Recall · AP · mAP · YOLOv8n live demo

Learning objectives

- Explain why classification ≠ detection, and what a detector outputs.
- Implement and interpret IoU and Non-Maximum Suppression; reason about confidence and NMS thresholds.
- Identify two-stage vs one-stage architectures; describe YOLO's grid intuition and anchor-free heads.
- Run YOLOv8n inference and read its raw output programmatically.

Disclosure: Claude was used to support organization, templating, coding, and formatting; all lecture ideas, content, adaptation, final decisions, and academic responsibility remain with the lecturer.

1

Why Object Detection?

From classification to localization

Why a single class label is not enough · Taxonomy of vision tasks

Lecture 9 → Lecture 10

Same backbone, different head — a natural extension of transfer learning.

L1 – L9 · IMAGE CLASSIFICATION

Image (H×W×3) → CNN backbone (pretrained) → Dense head → one class label

L10 · OBJECT DETECTION (today)

Image (H×W×3) → CNN backbone (same as before) → Detection head → N × (box, class, confidence)

The output head changes from a fixed-size class vector to a variable-size list of detections after post-processing.

Detection inherits the L1–L9 toolkit: pretrained backbones, augmentation, transfer learning.

Why Classification Is Not Enough

A real scene contains multiple objects at different positions and scales.

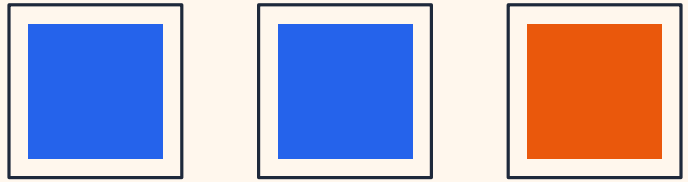
Whole-image classification

Input has 3 objects

Output: "dog"

One label, no location, no count

Object detection



```
class: dog    conf: 0.94    box: [142,  87, 512, 430]
class: dog    conf: 0.88    box: [560,  90, 890, 420]
class: person conf: 0.81    box: [680,  50, 930, 480]
```

What + where + how many

Classification = What. Detection = What + Where + How many.

Detection adds spatial reasoning to classification.

A Taxonomy of Vision Tasks

Same backbone — different output structures.

Task	Question	Output	Coverage
Classification	*What?*	`1 label`	Covered L1–L9
Localization	*What + where?*	`1 label + 1 box`	Introduced today
Detection	*What + where + how many?*	`N labels + N boxes`	TODAY
Semantic Segmentation	*What class per pixel?*	`Class mask (H×W)`	Next lecture
Instance Segmentation	*Which object per pixel?*	`Per-instance masks`	Beyond DATA305

All five tasks in the table can reuse a visual backbone — only the head differs.

L11 will cover semantic segmentation (U-Net).

2

Boxes · IoU · NMS

The geometry of detection

Bounding boxes · IoU · Confidence threshold · Non-maximum suppression

Bounding Boxes — What a Detection Contains

A detection is a 6-tuple: (class, confidence, x1, y1, x2, y2). Ultralytics YOLO exposes two coordinate formats.

Class label

What category? (e.g. 80 COCO classes)

```
0: person
1: bicycle
2: car   ...   78: hair drier
```

Confidence score

How sure is the model? In [0, 1].

```
0.91   high — likely correct
0.42   marginal — review threshold
```

Bounding box

Where is the object? Two common formats below.

```
[x1, y1, x2, y2]   pixels (corners)
[xc, yc,  w,  h]   normalized [0,1]
```

Reading YOLOv8 box formats programmatically

```
# Each detection in result.boxes carries both formats
x1, y1, x2, y2 = box.xyxy[0].tolist()    # corner format (pixels)
xc, yc, w, h   = box.xywhn[0].tolist()   # center format (normalized)
x1_back        = (xc - w/2) * W_img      # convert center → corner
```

Corner format is convenient for plotting; normalized format is convenient for resize-invariant comparisons.

Notebook §4.5 (read output) and §4.6 (coordinate conversion).

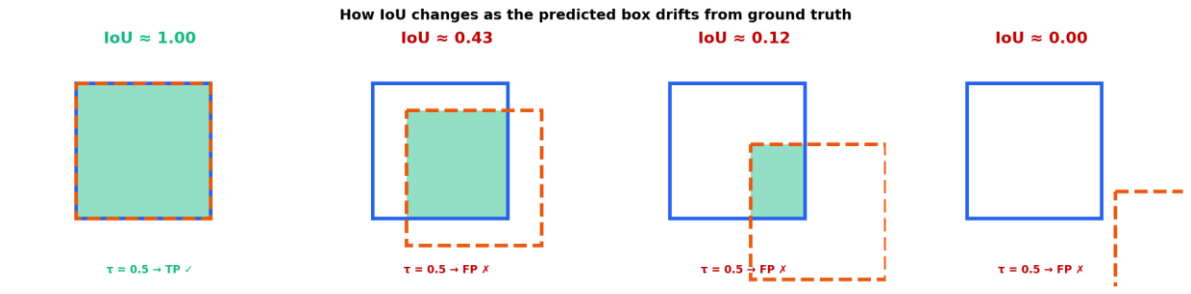
IoU — Intersection over Union

For a predicted box, TP/FP depends on both the class label and whether IoU exceeds the chosen threshold.

Definition

IoU = Area of Intersection / Area of Union

```
IoU = 0   →  no overlap
IoU = 1   →  perfect overlap
IoU ≥ 0.5 →  typical TP threshold (PASCAL VOC)
```



τ = 0.50

Standard threshold
PASCAL VOC AP@0.50 (AP50)

τ = 0.75

Strict threshold
Used in COCO AP@0.75

τ ∈ [0.50:0.05:0.95]

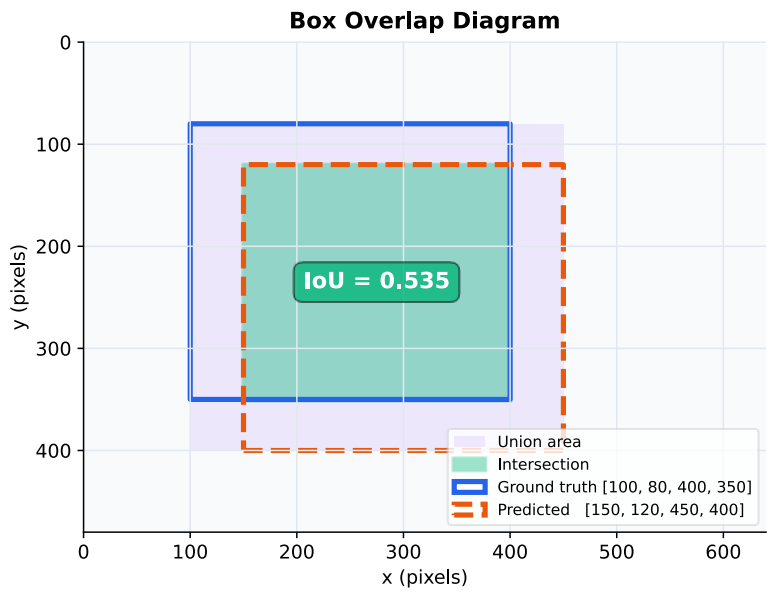
COCO mAP averages over 10 IoU thresholds — primary metric in modern benchmarks.

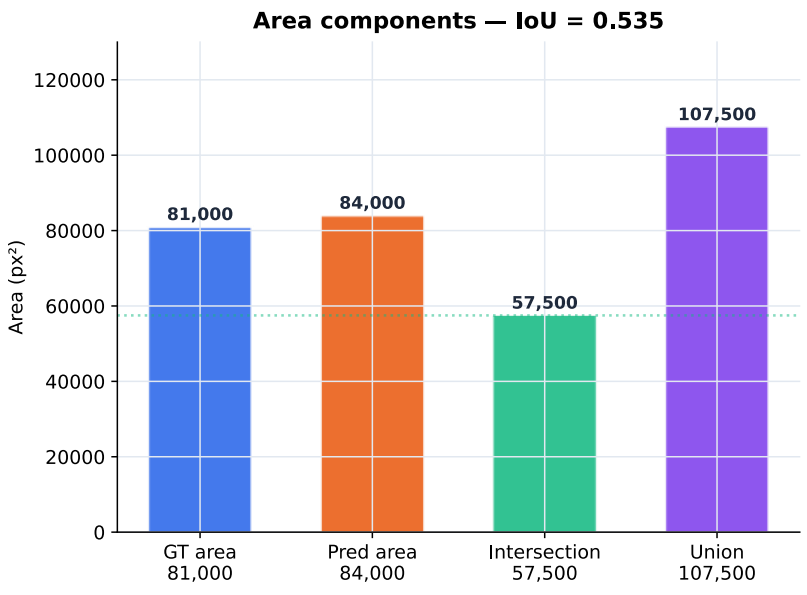
The same prediction can be a True Positive at τ=0.50 and a False Positive at τ=0.75.

Geron Ch. 14 Fig. 14-24 — IoU intuition. Notebook §2.1 / §2.2 — implementation.

IoU — Worked Example from the Notebook

Same prediction → different verdicts at different IoU thresholds.





IoU = 0.535: true positive at τ=0.50, false positive at τ=0.75.

Notebook output

GT box	[100, 80, 400, 350]
Predicted	[150, 120, 450, 400]
Intersection	57,500 px²
Union	107,500 px²
IoU	0.535

At τ = 0.50 → ✓ True Positive

At τ = 0.75 → ✗ False Positive

Notebook §2.2 — worked IoU example.

Confidence Threshold τ

A detector outputs many candidate boxes — τ decides which to keep.

Rule: keep box if confidence ≥ τ | discard otherwise

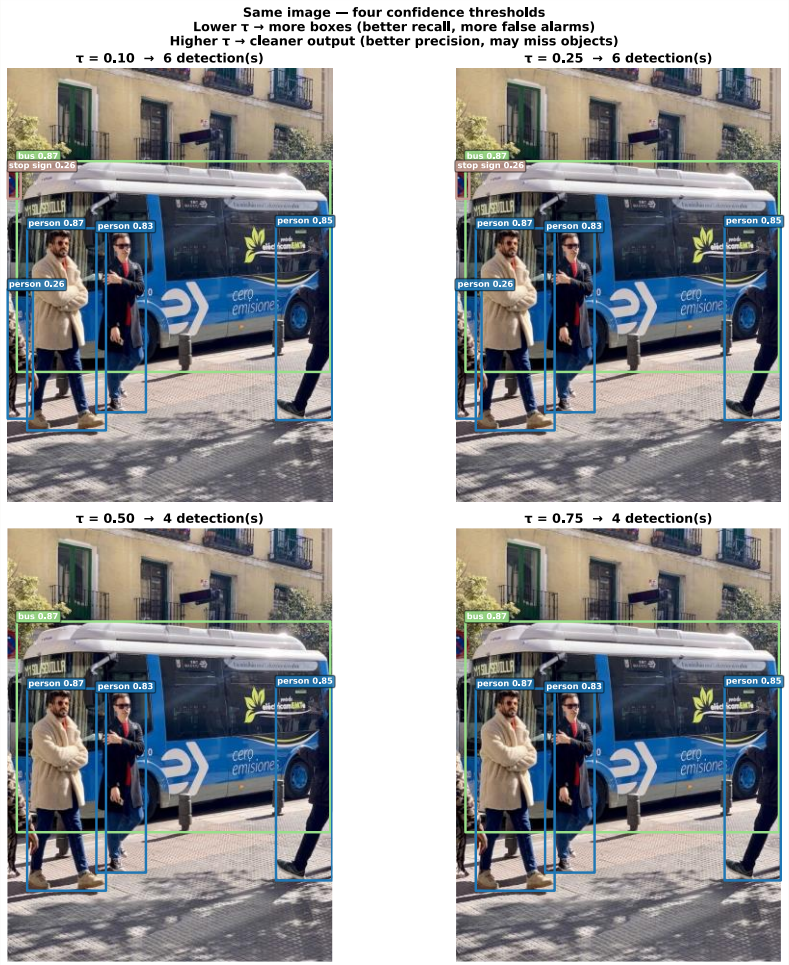
Low threshold (τ = 0.25)	High threshold (τ = 0.75)
✓ More objects recalled	✓ Fewer false positives
✓ Less likely to miss small / occluded objects	✓ Cleaner, more reliable output
✗ More false positives in busy scenes	✗ May miss low-confidence real objects
✗ Cluttered output at very low τ	✗ Worse recall on difficult images
Better recall — lower precision	**Better precision — lower recall**

In the demo we use τ = 0.25 (more recall) and τ = 0.50 (cleaner output). Always tune on a validation set.

Notebook §2.5 (simulated sweep); §4.7 for the actual bus.jpg threshold sweep.

Same Image — Four Confidence Thresholds

YOLOv8n on bus.jpg. As τ increases, weak detections drop off first.



What changes as τ increases?

τ = 0.10 — **6 detections**
Same set as τ=0.25 — no extra weak detections in this run

τ = 0.25 — **6 detections**
YOLOv8 default — bus + 4 persons + low-confidence stop sign

τ = 0.50 — **4 detections**
Stricter — drops the marginal far-side person and the stop sign

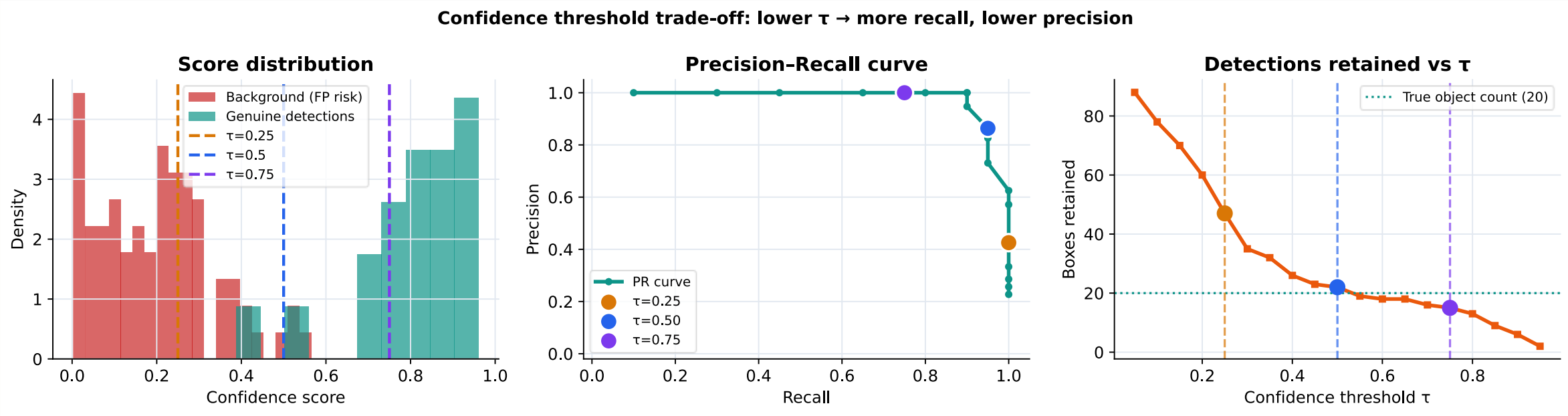
τ = 0.75 — **4 detections**
Very strict — only confident detections survive

Counts on bus.jpg: 6/6/4/4. In busier scenes very low τ creates more false positives.

Notebook §4.7 — bus.jpg at τ ∈ {0.10, 0.25, 0.50, 0.75}.

From One Threshold to a Precision–Recall Curve

Simulated confidence sweep — bus.jpg has no ground-truth labels, so we use a toy distribution to compute precision/recall.



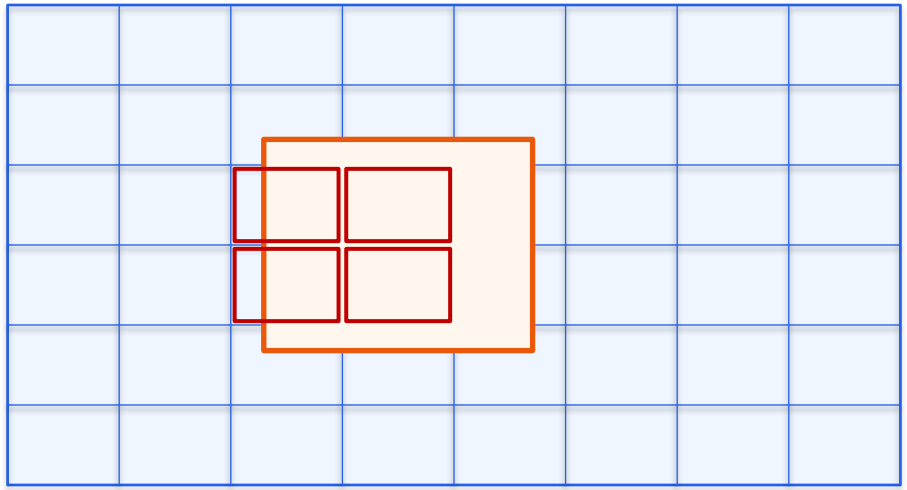
Each τ chooses one trade-off point. The PR curve summarizes the whole family — feeds AP and mAP.

Notebook §2.5 — simulated PR curve from a confidence-threshold sweep.

Why Do We Get Duplicate Boxes?

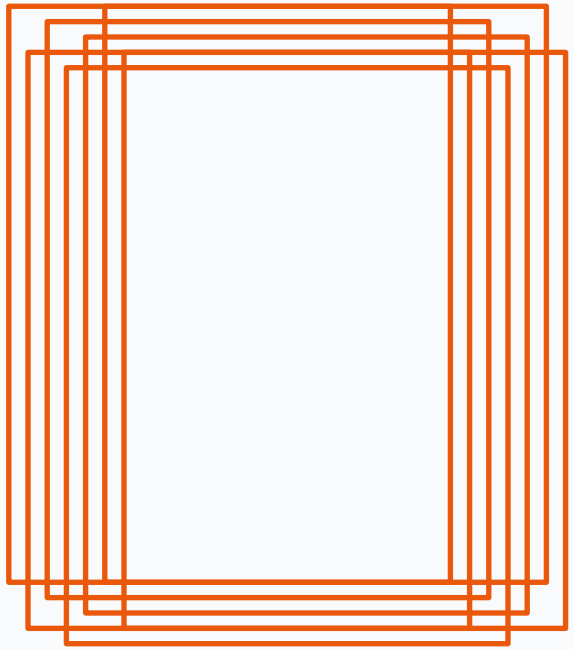
A real detector predicts on a dense grid — the same object fires at several nearby cells.

Dense candidate generation



Each cell predicts class scores and box offsets.
Several nearby cells may produce overlapping candidates
for the same object — that is why post-processing is needed.

Without NMS — many boxes per object



7 candidate boxes → 1 object

We need to collapse these duplicates — that's what Non-Maximum Suppression (NMS) does.

Geron Ch. 14 Fig. 14-25 — sliding / dense candidate generation.

Non-Maximum Suppression (NMS) — Algorithm

Greedy selection: keep the highest-confidence box, suppress its neighbors.

NMS algorithm (per class)

1. Sort all candidate boxes by confidence (descending).
2. Pick the top-confidence box → add to KEEP list.
3. Compute IoU between the picked box and all remaining boxes.
4. Remove any box with IoU ≥ iou_threshold.
5. Repeat steps 2–4 until the list is empty.

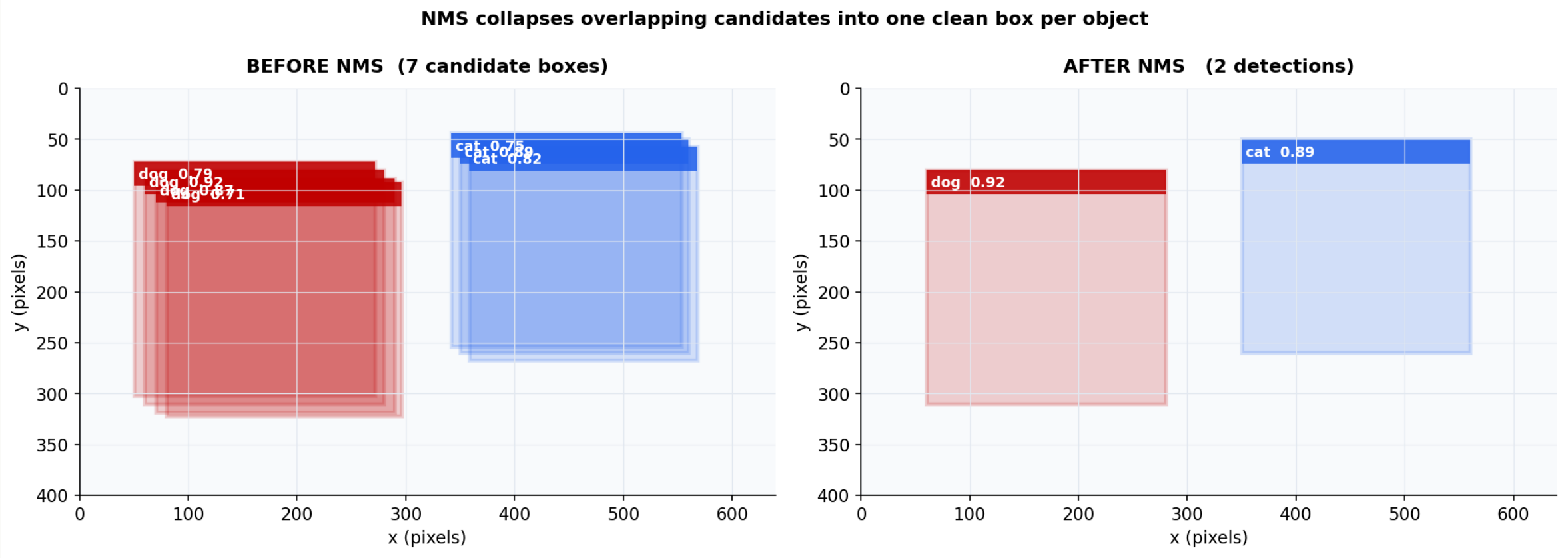
Core NMS for one class (notebook §2.6) — real detectors run this per class

```
def nms(boxes, scores, iou_threshold=0.45):
    order = np.argsort(scores)[::-1]
    keep  = []
    while order.size > 0:
        i = order[0]
        keep.append(int(i))
        if order.size == 1:
            break
        ious  = np.array([
            compute_iou(boxes[i], boxes[j])
            for j in order[1:]])
        order = order[1:][ious < iou_threshold]
    return keep
```

Real detectors run NMS independently per class — a dog and a cat overlapping heavily are never suppressed against each other.

Before vs. After NMS

Synthetic example: 7 candidate boxes (2 objects) — NMS collapses each cluster to one.

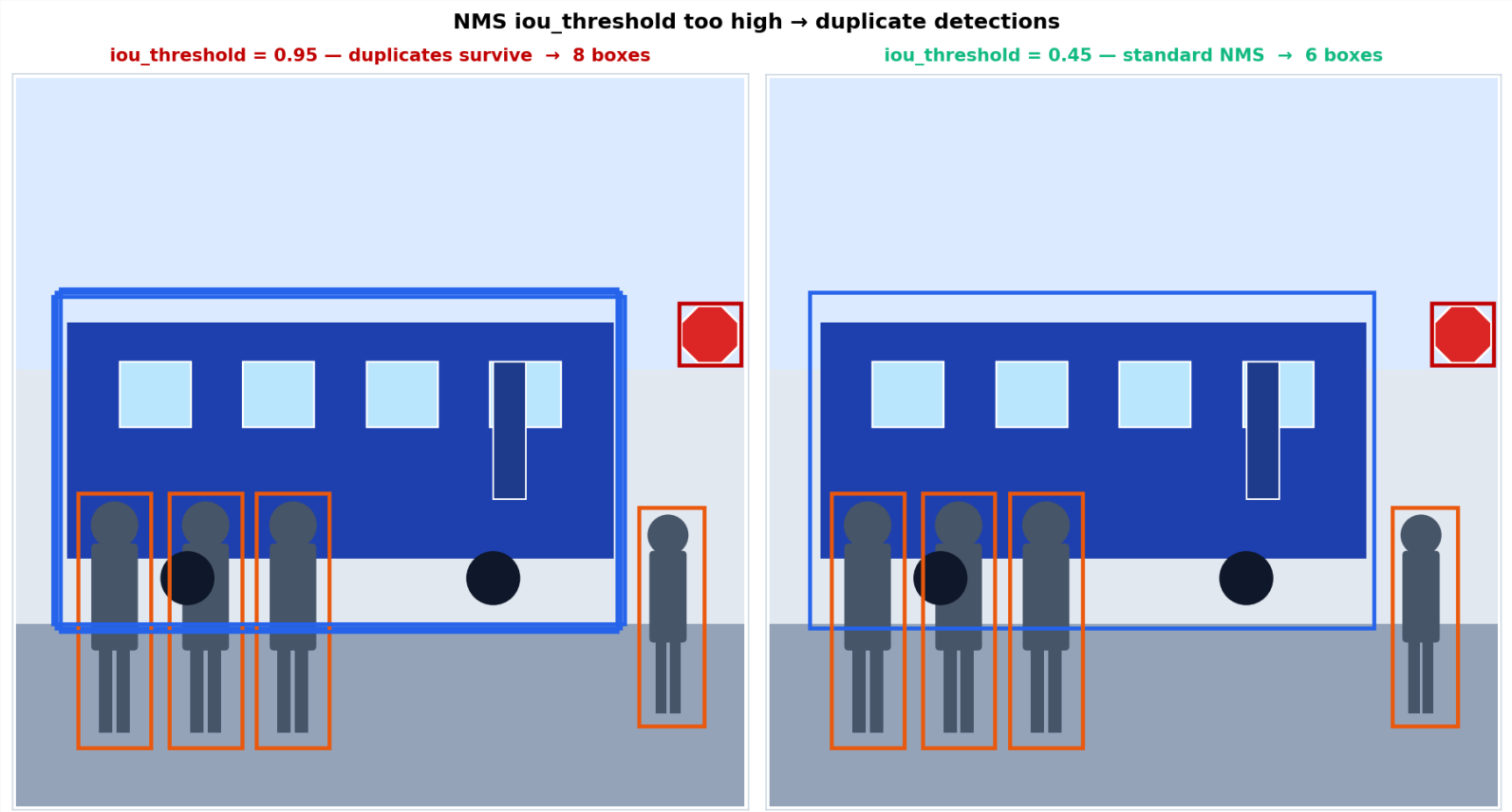


NMS collapses overlapping candidates into one clean box per object.

Notebook §2.8 — before vs after NMS visualization.

Failure Mode — NMS IoU Threshold Too High

If iou_threshold is too high, NMS barely suppresses anything → duplicate detections survive.

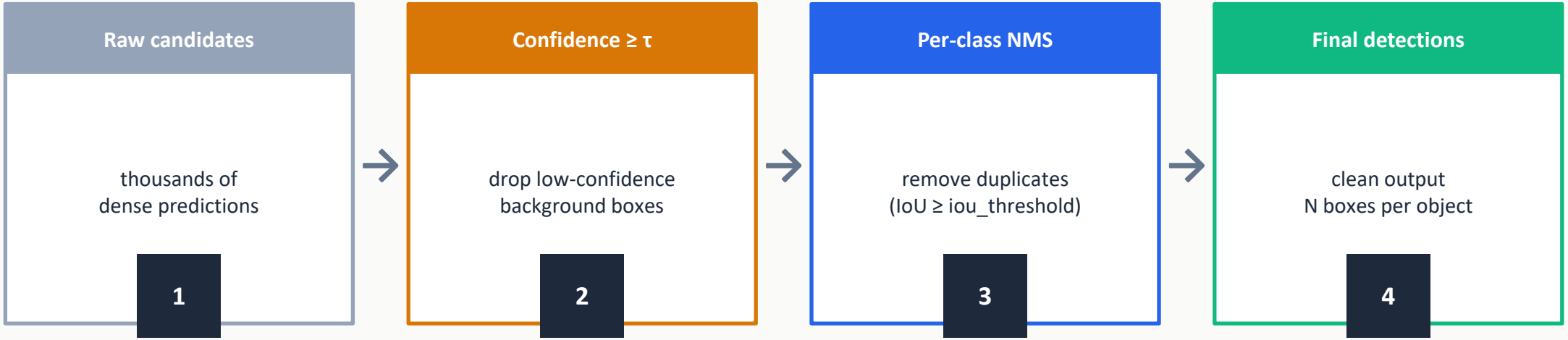


Default iou_threshold = 0.45 works for most cases. Raise carefully if you want to allow legitimately overlapping objects.

Notebook §4.9 — NMS iou_threshold = 0.95 (8 boxes) vs 0.45 (6 boxes).

Post-processing Pipeline

Two filters applied in order: confidence threshold, then NMS. Three stages: raw → filtered → final.



Conceptual flow — exact counts depend on model + image

Many	Fewer	One per object	Final
raw candidates	*after confidence filter*	*after NMS*	*detections output*

Inference output structure for any modern one-stage detector.

3

Detection Architectures

Two-stage vs. one-stage · R-CNN family · YOLO

How modern detectors are organized — and where the speed / accuracy trade-off comes from

Why Detection Is Harder Than Classification

Four core challenges that drove every architectural choice in modern detectors.

01

Variable number of outputs

Classification has 1 output. Detection has N — N is unknown a priori.

02

Scale variation

Objects can occupy 5% or 80% of the image. Feature maps must handle both.

03

Occlusion and clutter

Objects overlap, occlude each other, appear in front of complex backgrounds.

04

Joint localization + classification

Both head outputs (boxes and classes) must be trained together.

These challenges shape both the head design and the loss function of every modern detector.

Two-Stage Detectors — the R-CNN (Region-based CNN) Family

The history of two-stage detection in three iterations.

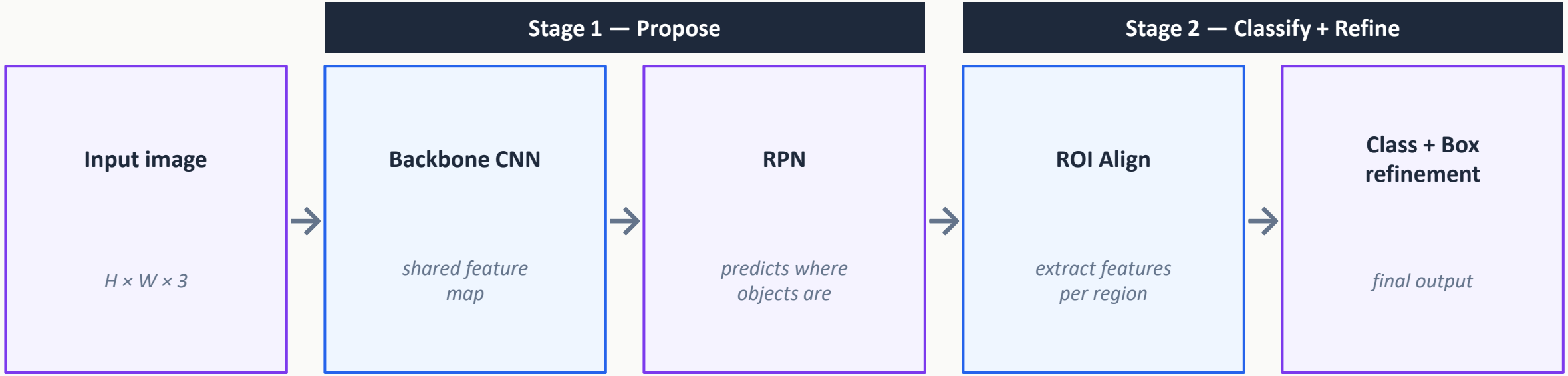
R-CNN *2014*	Region proposals + run a CNN on every cropped region.	**Slow — re-runs CNN per crop**
Fast R-CNN *2015*	Run CNN once on the whole image; share features via ROI pooling.	**Faster — shared backbone**
Faster R-CNN *2015*	Region Proposal Network (RPN) replaces selective search.	**End-to-end learnable**

Two-stage detectors typically prioritize localization accuracy over speed.

Geron Ch. 14 — Two-stage detectors (R-CNN family)

Faster R-CNN Pipeline (Simplified)

Two stages share the backbone — they predict together but at different abstraction levels.



Two-stage = propose first, classify later.

RPN (Region Proposal Network) narrows the search space to a few hundred candidate regions; the second stage refines them. Often more accurate but slower than comparable one-stage detectors; the exact gap depends on backbone, image size, and hardware.

Faster R-CNN — see Geron Ch. 14 for the full architecture diagram.

One-Stage Detectors — YOLO and SSD

Skip the proposal step — predict everything in one forward pass.

Core idea: dense feature-map locations predict boxes + classes directly in one forward pass.

SSD (Single Shot MultiBox Detector) (2016)

- Detects on multiple feature-map scales.
- Anchor-based — small, medium, large priors.
- Often faster than two-stage detectors; accuracy varies with backbone.
- Backbones: VGG, MobileNet, EfficientNet.

YOLO (You Only Look Once) family (2016 → present)

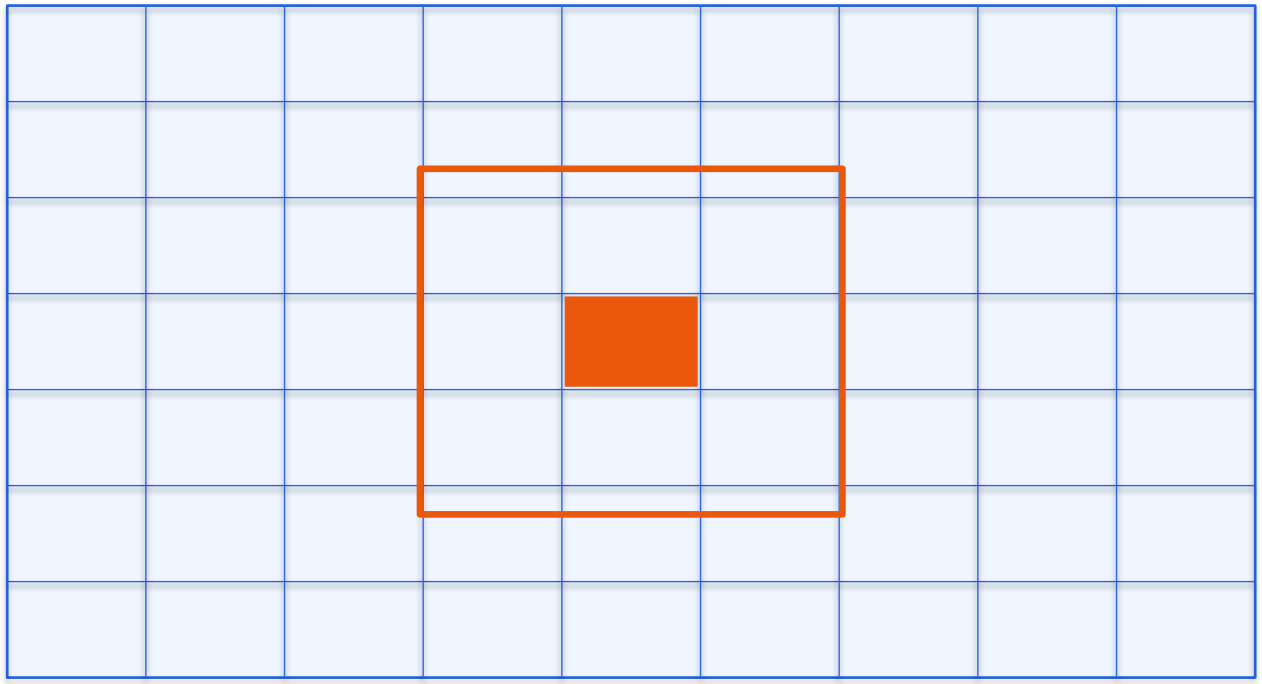
- Single forward pass — “you only look once”.
- YOLOv1: grid-based dense prediction (no anchor boxes).
- YOLOv2–v5: anchor-based priors introduced.
- YOLOv8+: anchor-free heads, pretrained on COCO.

One-stage detectors typically optimize for speed. Modern variants (YOLOv8) approach two-stage accuracy.

YOLO Intuition — One Forward Pass

Classic YOLO grid view — modern YOLOv8 still works on dense feature-map locations, but is anchor-free.

Classic YOLO grid concept



Each cell predicts: B boxes · objectness · class probabilities

Cell whose center contains the object is responsible for that detection.

Why this works

- All predictions in one forward pass — often faster than proposal-based pipelines.
- Spatial reasoning is built into the architecture: nearby cells share context via the convolutional backbone.
- Output tensor has fixed shape — easy to train end-to-end.
- YOLOv8 keeps the dense-prediction philosophy but removes anchor priors.

Custom YOLO grid illustration based on the discussion after Geron Ch. 14.

Anchor Boxes — Concept and Caveat

Historical: anchor templates. Modern (YOLOv8): anchor-free direct prediction.

Anchor-based (Faster R-CNN, SSD, YOLOv3–v5)

How it works

```
location + anchor template  →  predict offsets
```

- Pre-defined boxes with different aspect ratios and scales.
- Each location predicts offsets relative to the anchor.
- Easier regression — small offsets from a known prior.
- Anchor sizes are hyperparameters that need tuning.

Anchor-free (YOLOv8, FCOS, CenterNet)

How it works

```
location  →  predict center + size directly
```

- No anchors — each location predicts box center + width + height.
- Simpler to train; fewer hyperparameters to tune.
- Same idea as FCOS / CenterNet.
- Recent Ultralytics YOLO releases use anchor-free heads.

Anchor-based remains widely deployed; anchor-free simplifies training and removes anchor tuning.

Recent Ultralytics YOLO releases use anchor-free heads.

Choosing a Detector

Match the architecture to your speed / accuracy / hardware constraints.

Need maximum accuracy

Two-stage: Faster R-CNN.

- Compute is available — GPU, batch size > 1, latency budget OK.
- Domain: medical imaging, satellite imagery, scientific analysis.

Need real-time inference

One-stage: YOLOv8, YOLOv11.

- Latency-bounded use cases — robotics, video, dashcam.
- YOLOv8n: 3.2M params · CPU inference is feasible for small demos; benchmark for real-time use.

Custom domain, fine-tune

Pretrained backbone + new head.

- Few annotated images — leverage transfer learning.
- YOLO supports `model.train(data=<your.yaml>)` out of the box.

Edge / mobile deployment depends on optimization (TensorRT, ONNX, INT8) and hardware — start with YOLOv8n + benchmark.

Practical guidance — verify with your own latency / accuracy benchmarks.

4

Evaluation & YOLOv8 Demo

Precision · Recall · AP · mAP · live inference

How detection is scored — and what YOLOv8 actually outputs

Evaluation — Precision, Recall, AP, mAP

How detection accuracy is scored. AP and mAP build on precision and recall.

Precision

$$TP \quad / \quad (TP + FP)$$

Of all predicted boxes, how many were actually correct?

Recall

$$TP \quad / \quad (TP + FN)$$

Of all real objects, how many did the model find?

Average Precision (AP)

Per-class score — summarizes the PR curve.

Behaves like the area under a smoothed / interpolated PR curve. Computed at one IoU threshold (e.g. AP50) or averaged over many (COCO AP@[0.50:0.05:0.95]).

Mean Average Precision (mAP)

Average AP across all classes — single overall score.

mAP@0.50 = mean over classes at IoU=0.50 (PASCAL VOC). mAP@[.50:.95] = COCO primary — averages across IoU thresholds AND classes.

AP / mAP are intuitively the area under a smoothed PR curve — but the formal definition uses precision interpolation.

Geron Ch. 14 — AP and mAP definitions.

Practical Demo — YOLOv8 Inference Setup

No training. Load a pretrained model, point it at an image, read the output.

1

Install ultralytics

```
pip install ultralytics  (Ultralytics package; GPU not
required; Internet required for first package/model download.)
```

2

Load pretrained model

```
model = YOLO('yolov8n.pt')   # 3.2 M params, 80 COCO classes
```

3

Run inference

```
results = model(image, conf=0.25, iou=0.45)
```

4

Read the output

```
for box in results[0].boxes:   # (cls, conf, xyxy, xywhn)
```

Minimal working example

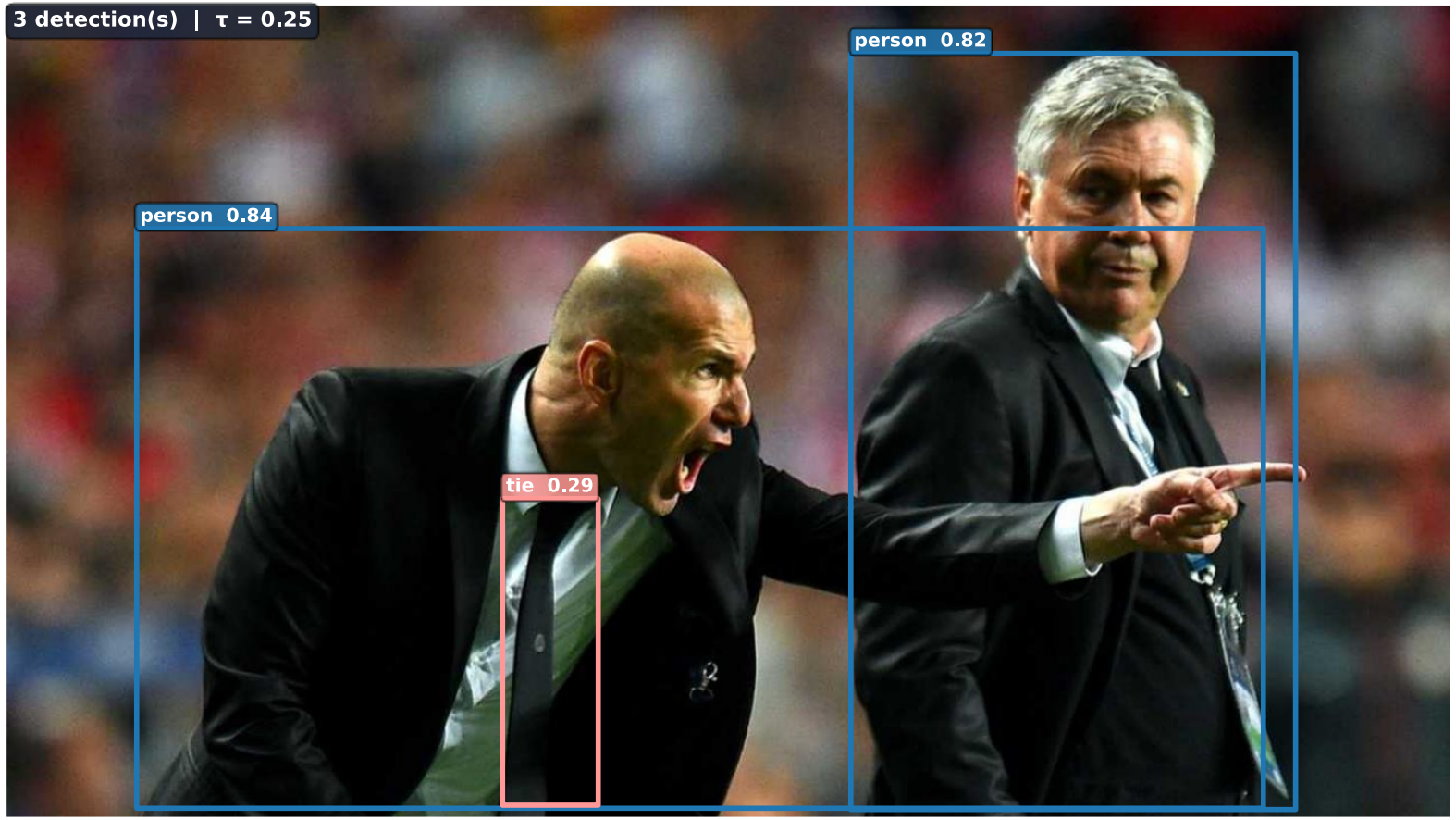
```python
from ultralytics import YOLO

model = YOLO('yolov8n.pt')

results = model(
    'bus.jpg',
    conf=0.25,        # confidence threshold
    iou=0.45,         # NMS IoU threshold
)

for box in results[0].boxes:
    cls   = int(box.cls)
    conf  = float(box.conf)
    coords = box.xyxy[0].tolist()
    print(model.names[cls], conf, coords)
```

Notebook §4.1 (load model); §4.5 (read output).

Demo Results — YOLOv8n on Simple & Dense Scenes

Two test images: simple sports scene vs. dense street scene.

YOLOv8n — zidane.jpg (simple scene, conf ≥ 0.25, iou=0.45)



3 detections — 2 persons + 1 lower-confidence tie. Main objects are found cleanly.

YOLOv8n — bus.jpg (dense scene, conf ≥ 0.25, iou=0.45)

6 detections — bus, 4 persons, low-confidence stop sign.

Failure Cases & Threshold Tuning

Knowing when a detector fails is as important as knowing when it succeeds.

τ too high	τ too low	NMS IoU too high	Very small / occluded objects
misses real objects	*may keep weak / spurious detections in busier scenes*	*duplicates survive*	*missed regardless of τ — needs higher resolution or specialized head*

Right thresholds depend on the validation set and the cost of missed vs spurious detections. Notebook §4.9–4.10.

Summary — Lecture 10

Five takeaways and a forward link to Lecture 11.

01

Detection = What + Where + How many

A single class label is never enough for a real scene.

02

IoU is the localization metric

TP / FP decisions depend on it; AP50 uses IoU ≥ 0.50 — COCO also averages stricter thresholds.

03

τ controls precision–recall trade-off

Lowering τ recovers objects at the cost of more false positives.

04

NMS removes duplicates

Dense candidate prediction → per-class greedy suppression by IoU.

05

YOLOv8n: practical inference today

yolov8n.pt ≈ 6 MB weights, 80 COCO classes, anchor-free, CPU-friendly for small demos.

Next → Lecture 11: Image Segmentation (U-Net) — pixel-level prediction on Oxford Pets trimap masks.